

Ambient drone by Time stretch

Time stretch algorithm and ambient music

kotan.kn @ 2025 December 13th



Purpose

About this lecture

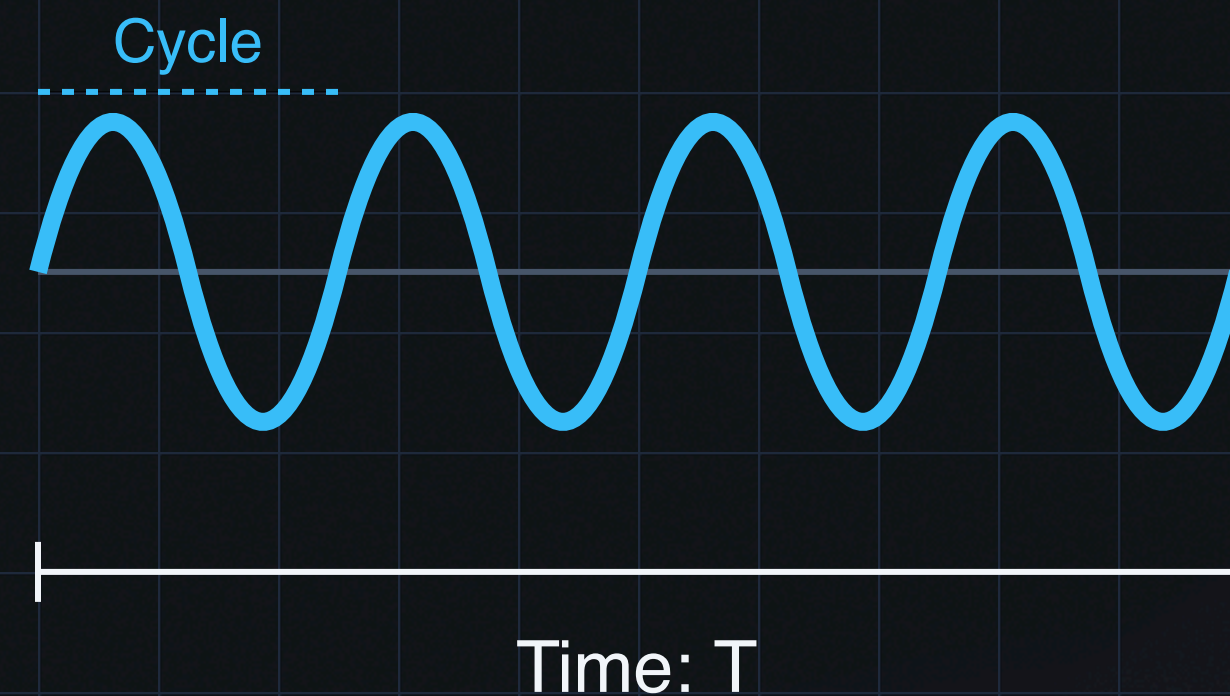
- Understand textbook time stretch algorithms
- Implement time stretch with Jitter
- Synthesize ambient drone sound by time stretch algorithms
- Introduce and implement a new approach of time stretch to maintain phase coherent
- Application of the algorithm, cross-synthesis and granular

Examples

<https://soundcloud.com/kotan-kn/sets/ambient>

To Time Stretch of Signal Duration

Original Signal



Play at 0.5x Speed

(No OLA/PV)

Stretched Signal



Pitch Drops!

Original (High Pitch)



Resampled (Low Pitch)



"Chipmunk Effect" (reversed)

Why OLA/PV is needed:

To stretch time stretching the waveform itself.
(Preserving the frequency content)

Time Stretch Requirement

Input Signal

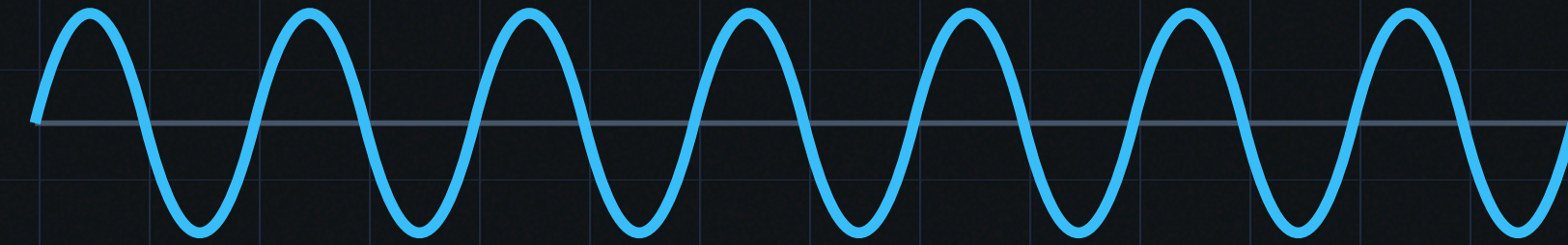


Original Time (T)

Time Stretch



Output Signal



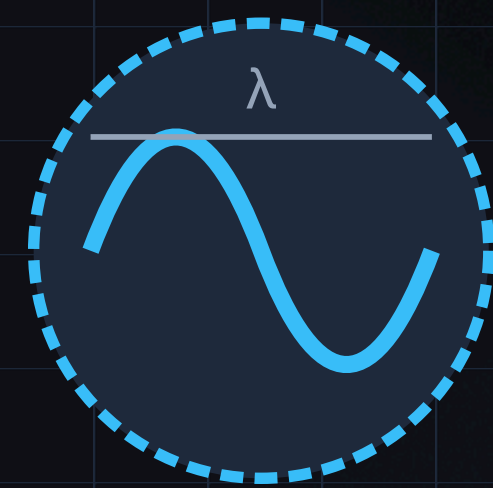
Stretched Time (αT)



Input Pitch

=

Frequency Conserved



Output Pitch

Key Point

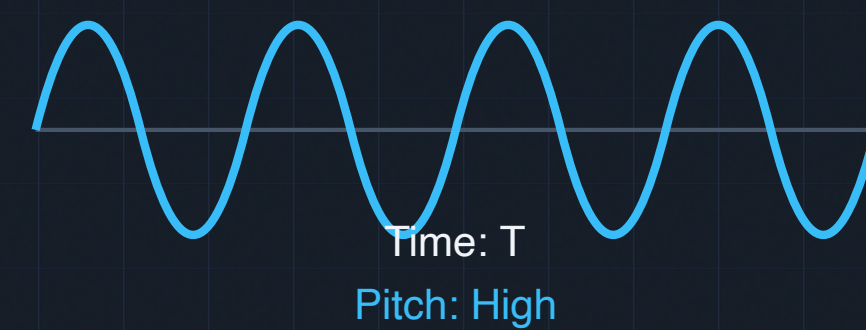
Waveform is elongated in time,
BUT the oscillation speed (pitch) stays the same.

Time Stretch

Method A: Resampling (Simple Speed Change)

Slowing down playback stretches the waveform, lowering the pitch.

Original



0.5x Speed

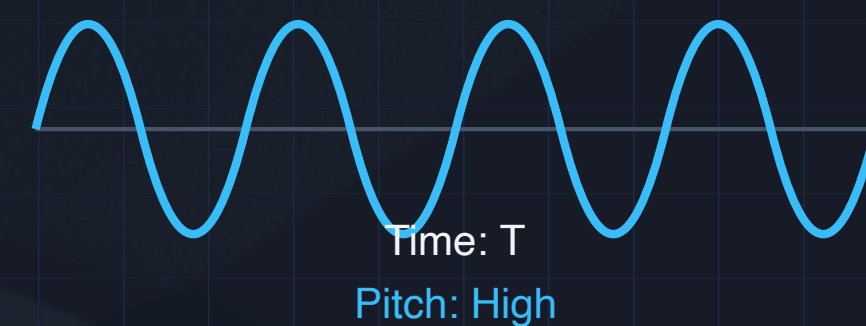
Result



Method B: Time Stretch (OLA / Phase Vocoder)

Signal length is extended, but frequency content (pitch) is preserved.

Original



Stretch 200%

Result



Resampling

Time ↑ = Pitch ↓
Like slowing down a vinyl record.

Time Stretch

Time ↑ = Pitch → (Unchanged)
Uses algorithms (OLA, PV) to repeat grains.

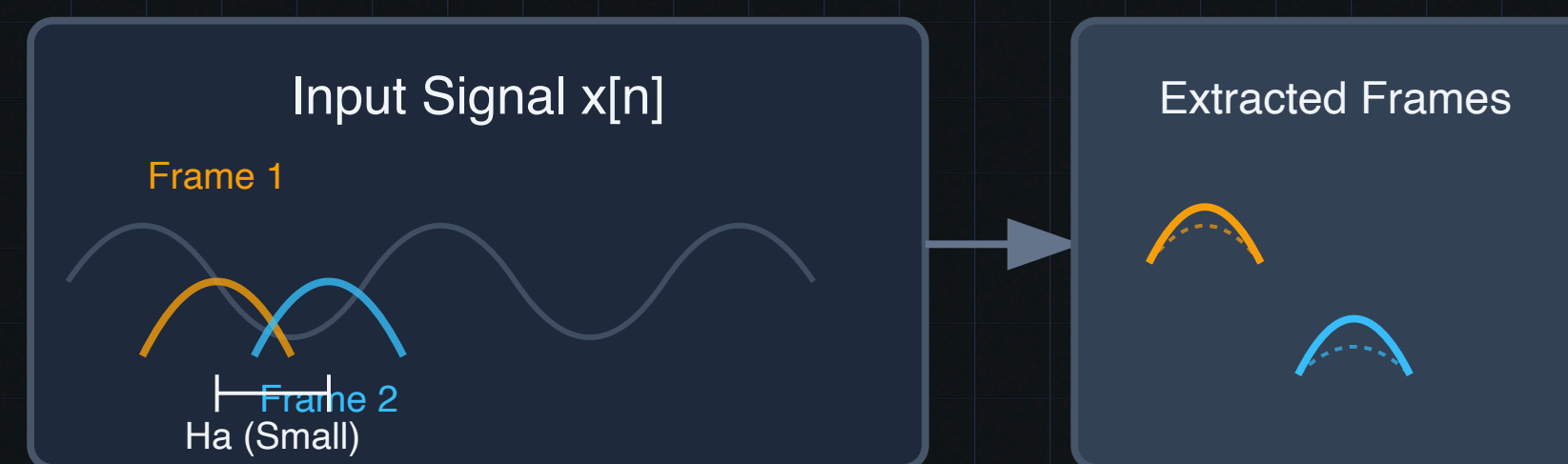


Overlap-Add algorithm

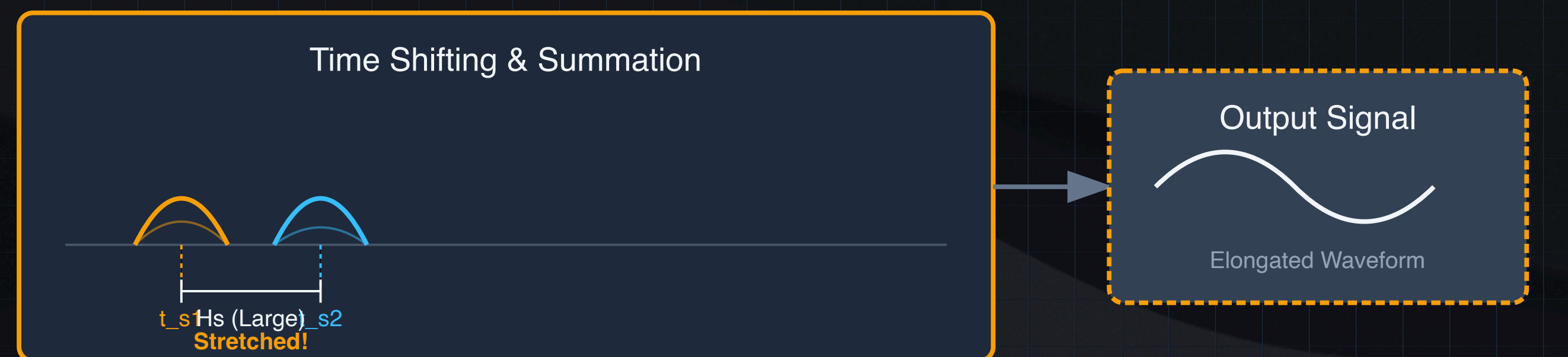
Kernel of Time Stretch Algorithms

- Window from source signal
- Merge windowed signal with in sliding sequences
- $y_0(t) = \emptyset$
- $y_{k+1}(t) = y_k(t) + w_k(t)x(\alpha_k t + \beta_k)$:
whole synthesised waveform at step k

1. Analysis Stage (Extraction)



2. Synthesis Stage (Overlap-Add)



Time Stretch Logic

- Frame 1 (Start)
- Frame 2 (Shifted)
- $H_s > H_a \rightarrow \text{Stretch}$

Demonstration/ Experiment

OLA-0.maxpat, OLA-1.maxpat

OLA and Phase Coherent

- Vanilla OLA does not maintain phase coherent and causes 'artifact' sound
- $y_{k+1}(t) = w_{k+1}(t)x(t + \Delta t) + w_k(t)x(t) + y_{k-1}(t)$
- $x(t) + x(t + \Delta t_k)$: Interference gain depends on their frequency
 - i.e. Completely silent all frequencies $\frac{n}{2\Delta_t}$

Phase coherent of Overlap-Add

Standard OLA (No Phase Compensation)

Phase is copied directly. Shifts cause peaks to align with troughs.



Horizontal Phase Coherence Loss: Waveform continuity is broken at frame boundaries.

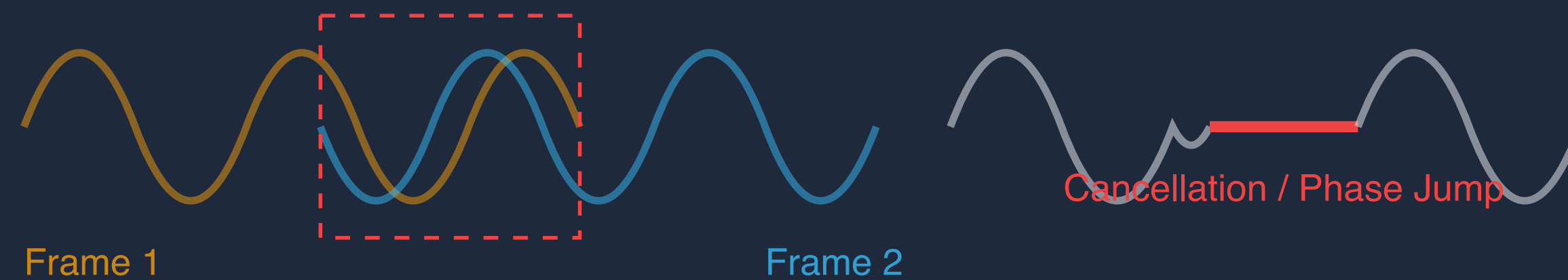
Demonstration/ Experiment

OLA-0.maxpat, OLA-1.maxpat

To restore phase coherence

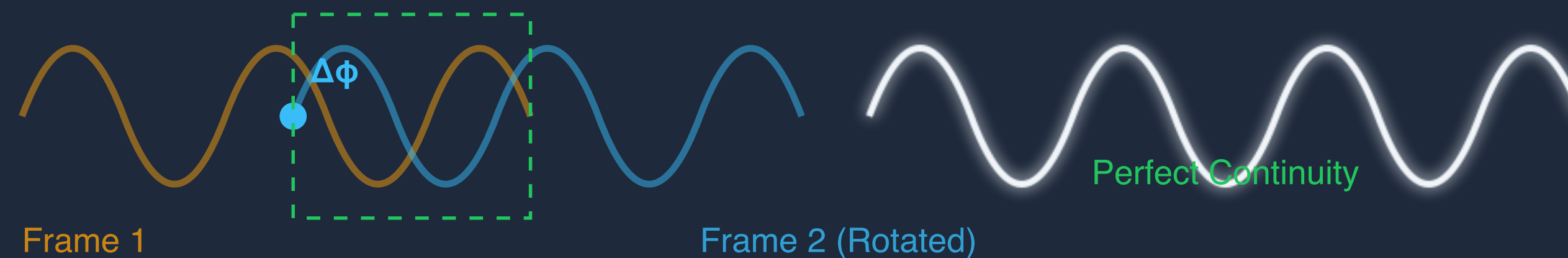
1. Standard OLA (No Compensation)

Phase is simply copied. Time shift causes mismatch



2. Phase Vocoder (Phase Locked)

Phase is rotated to match the expected arrival time



Horizontal Coherence: Continuity of a single frequency component over time.

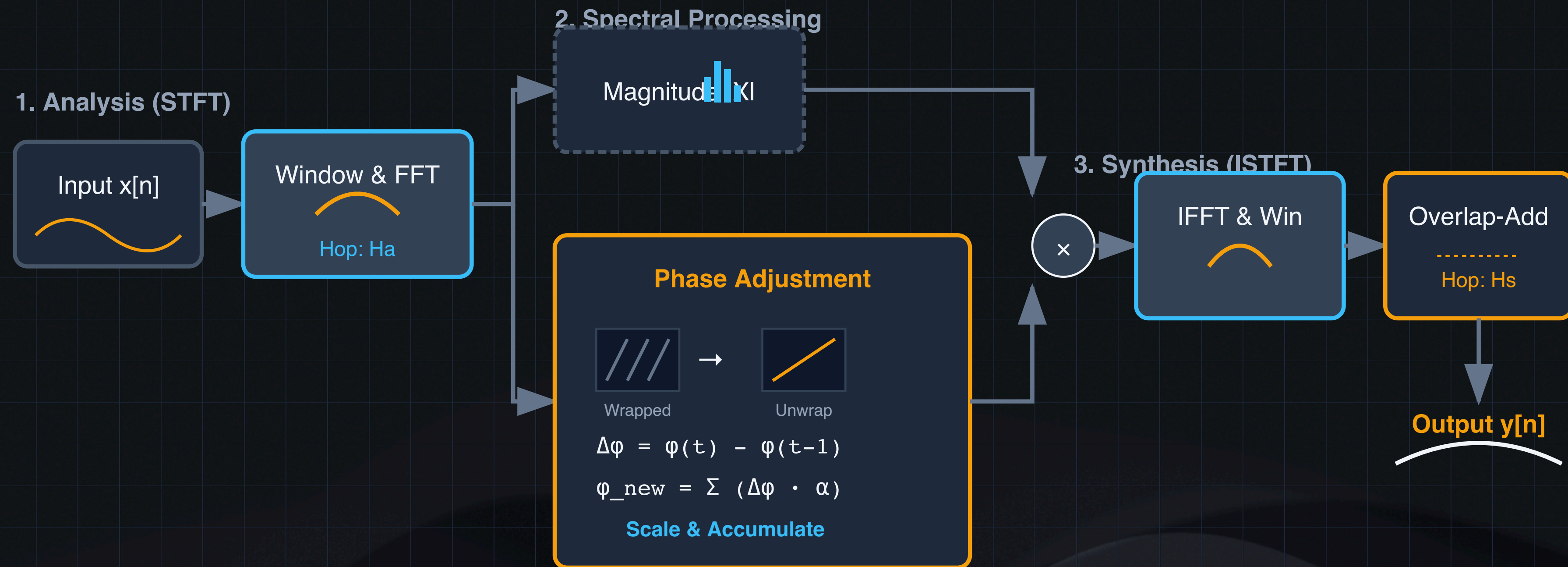
[handwrite figure]

Traditional Phase Vocoder

Maintain horizontal phase coherent

- Adjust phase in frequency domain by rotating exponentials
- $y_{k+1}(t) = y_k(t) + w_k(t) \mathcal{F}^{-1} [D_k(\omega; t)]$: Whole synthesised signal at k-step
- $D_k(\omega; t) = \|X_k(\omega; t)\| \exp(-2\pi j \theta_k(\omega))$: Synthesised frame at k
- $X(\omega; t) = \mathcal{F} [w_k(t) x(\alpha_k t + \beta_k)]$: Source STFT spectrum
- $\theta_{k+1}(\omega) = \theta_k(\omega) + \alpha \omega [\angle X_{k+1}(\theta; t) - \angle X_k(\theta; t)]$: Accumulated phase progress

Traditional Phase Vocoder



Phase Coherence

Phase is adjusted to match the new time grid (H_s) preserving instantaneous frequency.

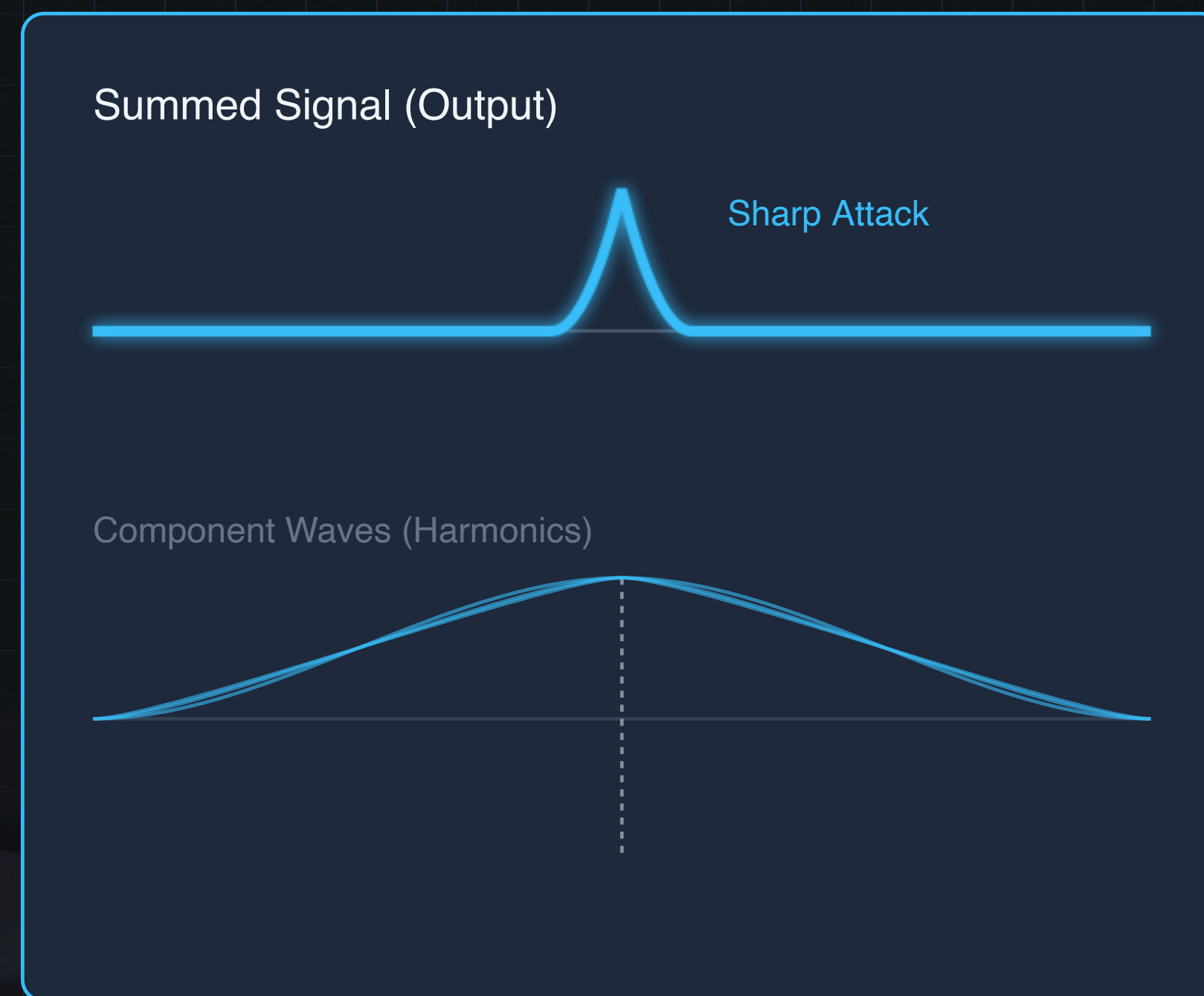
Demonstration/ Experiment

PhaseVocoder-1.maxpat

Vertical Phase Coherent

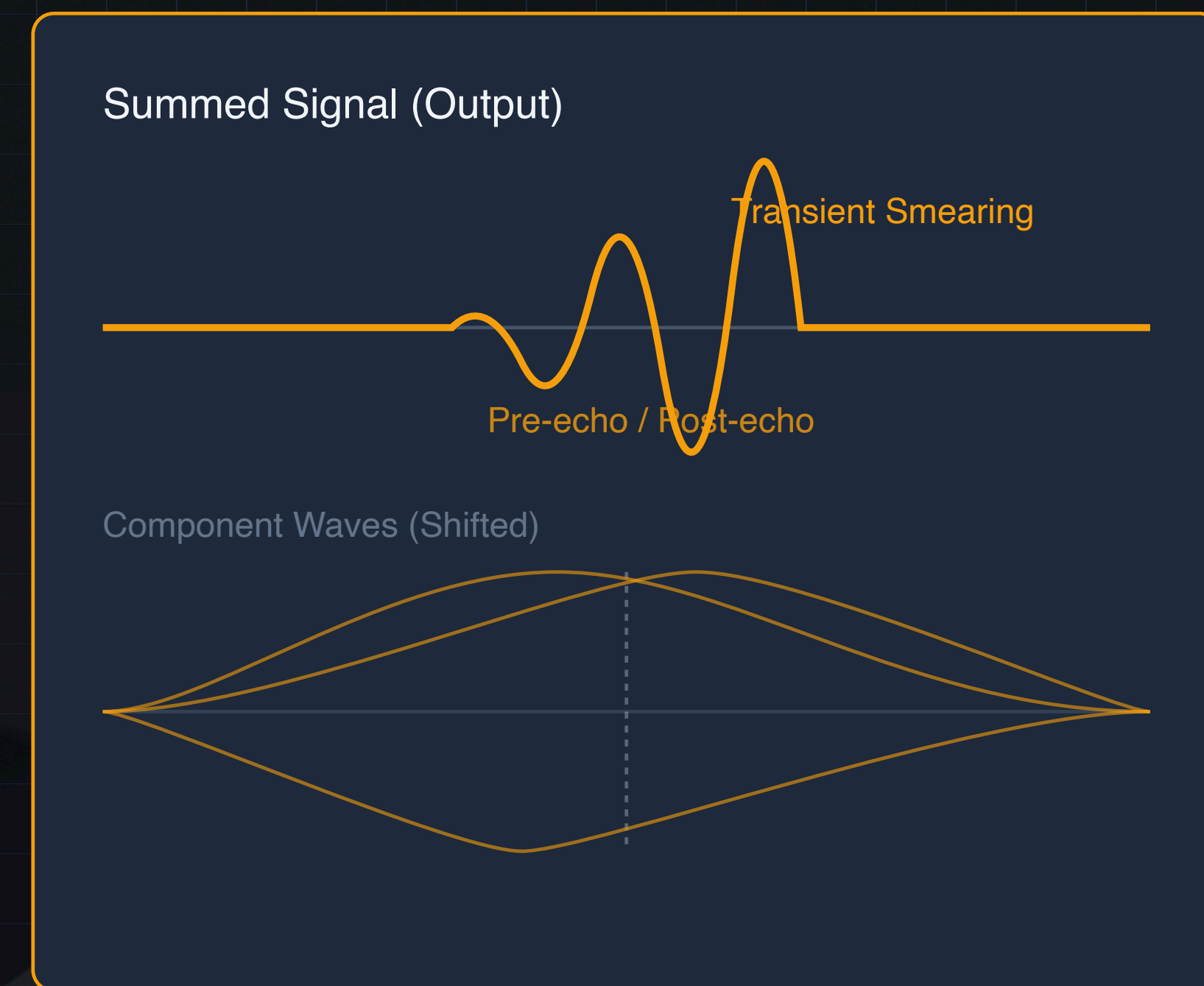
Phase Locked (Sharp)

All frequencies align at transient time



Phase Scrambled (Smeared)

Random phase shifts disperse energy



Vertical Phase Coherence is crucial for preserving percussive sounds (transients).

A new approach to maintain
phase coherent among frames

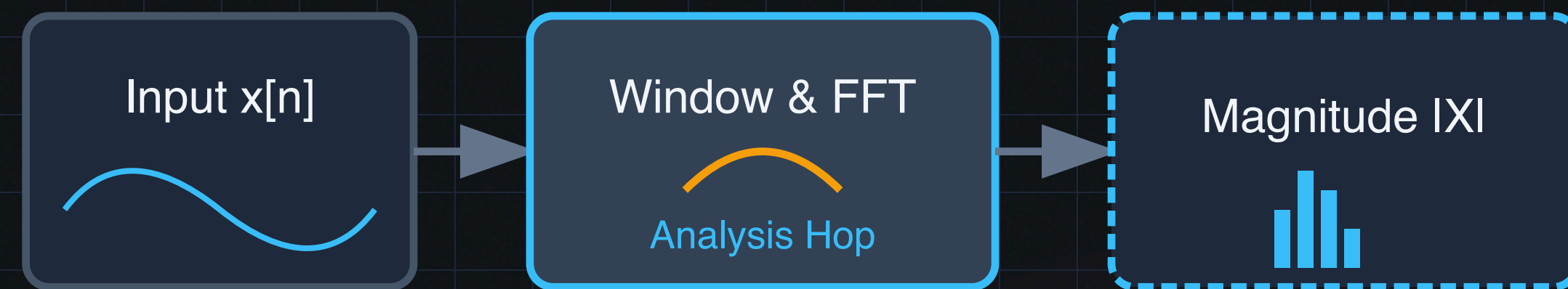
Phase Connect Vocoder

Reuse phase spectrum from OLA waveform

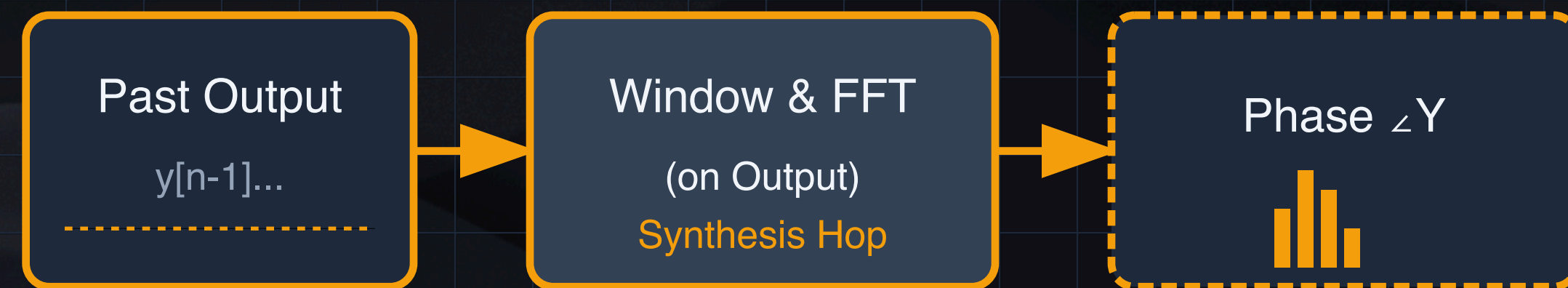
- Refer phase from synthesised signal frame
- $y_{k+1}(t) = y_k(t) + w_k(t) \mathcal{F}^{-1} [D_k(\omega; t)]$: Whole synthesised signal at k-step
- $D_k(\omega; t) = \|X_k(\omega; t)\| \exp(2\pi j \angle Y_k(\omega; t))$: Synthesised signal frame
- $X(\omega; t) = \mathcal{F} [w_k(t) x(\alpha_k t + \beta_k)]$: Source spectrum
- $Y_k(\omega; t) = \mathcal{F} [w_k(t) y_k(t)]$: Estimated Target phase

Phase Connect Vocoder

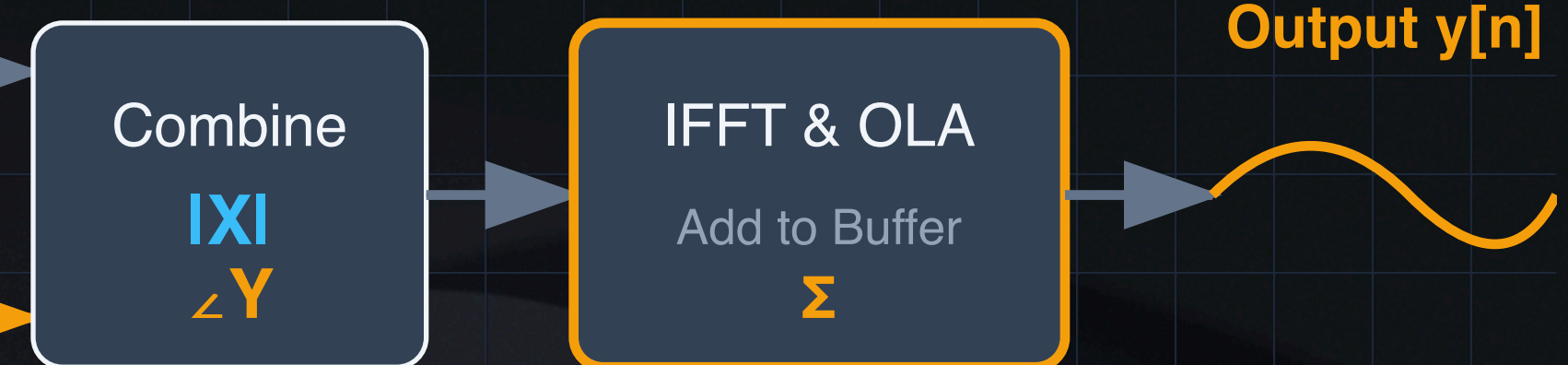
1. Amplitude Estimation (Input)



2. Phase Estimation (Feedback)



3. Synthesis



Why this works?

- Amplitude preserves timbre (Input)
- Phase ensures continuity (Output)
- **Reduces Phasiness**

Feedback Loop (Output becomes next Phase Reference)

Tight-width Hann Window Function

- Focus on existing waveform to estimate suitable phase
- Weighted Half-Size hann window
- Traditional Hann; $W(n) = \frac{1}{2} - \frac{1}{2} \cos \left(2\pi \frac{n}{N} \right)$ for $0 \leq n < N$
- Modified Hann: $W(n) = \frac{1}{2} - \frac{1}{2} \cos \left(2\pi \min(1, \frac{2n}{N}) \right)$ for $0 \leq n < N$

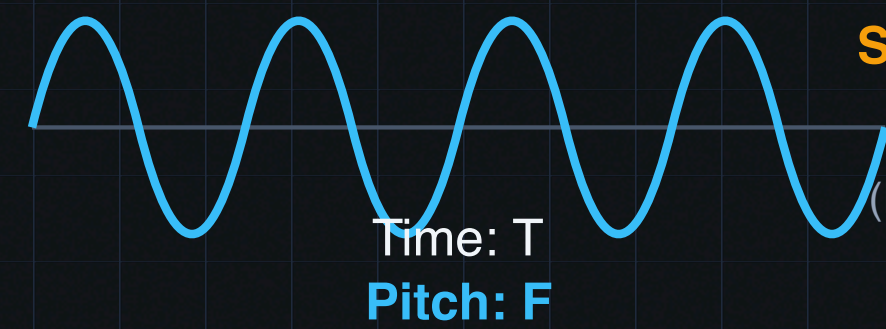
[Demonstration]

And more application

Application

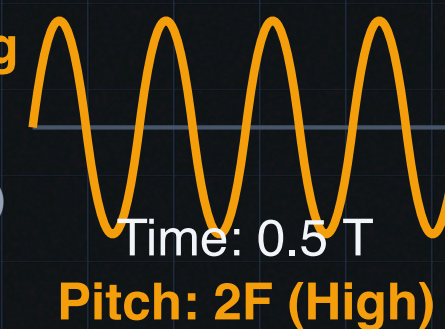
Pitch shift

1. Original Signal



Step A: Resampling
(Play Faster / 2x Speed)

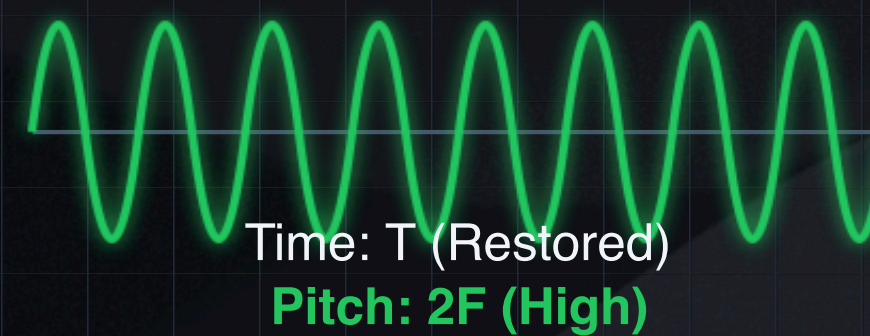
2. Speed Changed



Side Effect: Duration is halved
(Need to restore length)

Step B: Time Stretch
(Stretch 200% via OLA/PV)

3. Final Output (Pitch Shifted)



Summary: The 2-Step Process

Original	Resample	Intermediate	Stretch	Result
Length: L	→	Length: L / k	→	Length: L (Fixed!)
Pitch: P		Pitch: P × k		Pitch: P × k

Analogy: Playing a tape faster (Chipmunk voice)
+ Copy-pasting segments to fill the silence.

Application

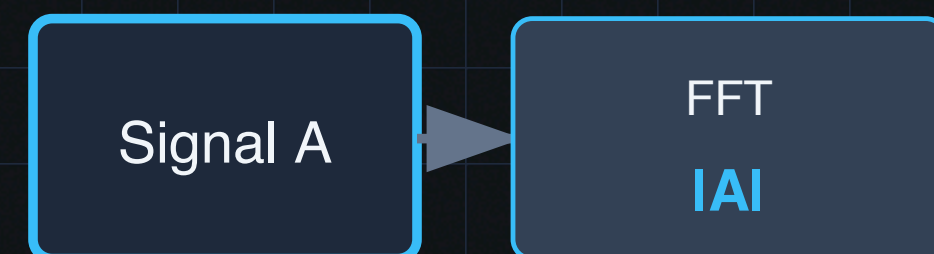
Cross synthesis

- Phase connect vocoder can contain horizontal phase with any magnitude spectrum
- $y_{k+1}(t) = y_k(t) + w_k(t) \mathcal{F}^{-1} [D_k(\omega; t)]$: Whole synthesised signal at k-step
- $D_k(\omega; t) = \|X_{0,k}(\omega; t)\| \|X_{1,k}(\omega; t)\| \exp(2\pi j \angle Y_k(\omega; t))$: Synthesised signal frame
- $X_{0,k}(\omega; t) = \mathcal{F} [w_k(t) x_{0,k}(\alpha_k t + \beta_k)]$, $X_{1,k}(\omega; t) = \mathcal{F} [w_k(t) x_{1,k}(\alpha_k t + \beta_k)]$: Source
- $Y_k(\omega; t) = \mathcal{F} [w_k(t) y_k(t)]$: Estimated Target phase

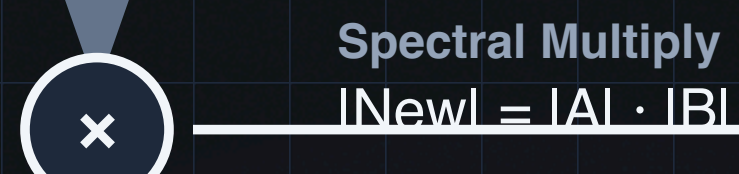
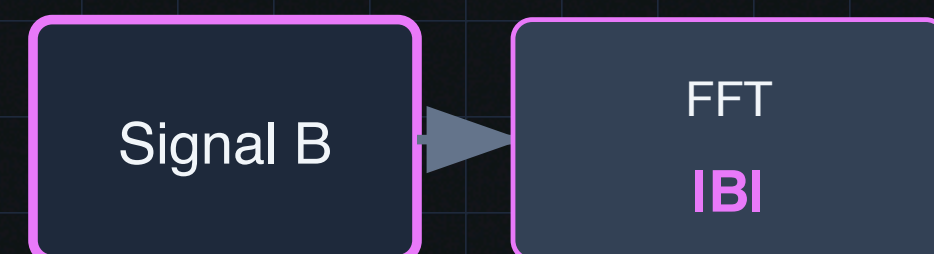
Application

Cross synthesis

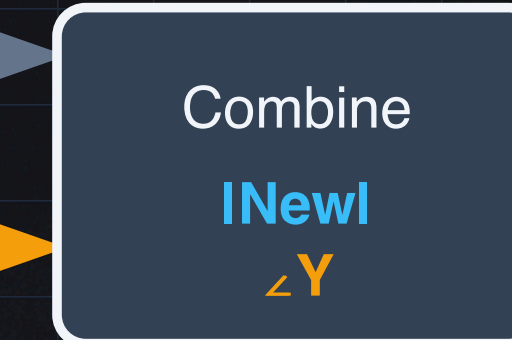
Input A (e.g. Voice)



Input B (e.g. Synth)



Synthesis Stage



Output

Phase Feedback (Output)



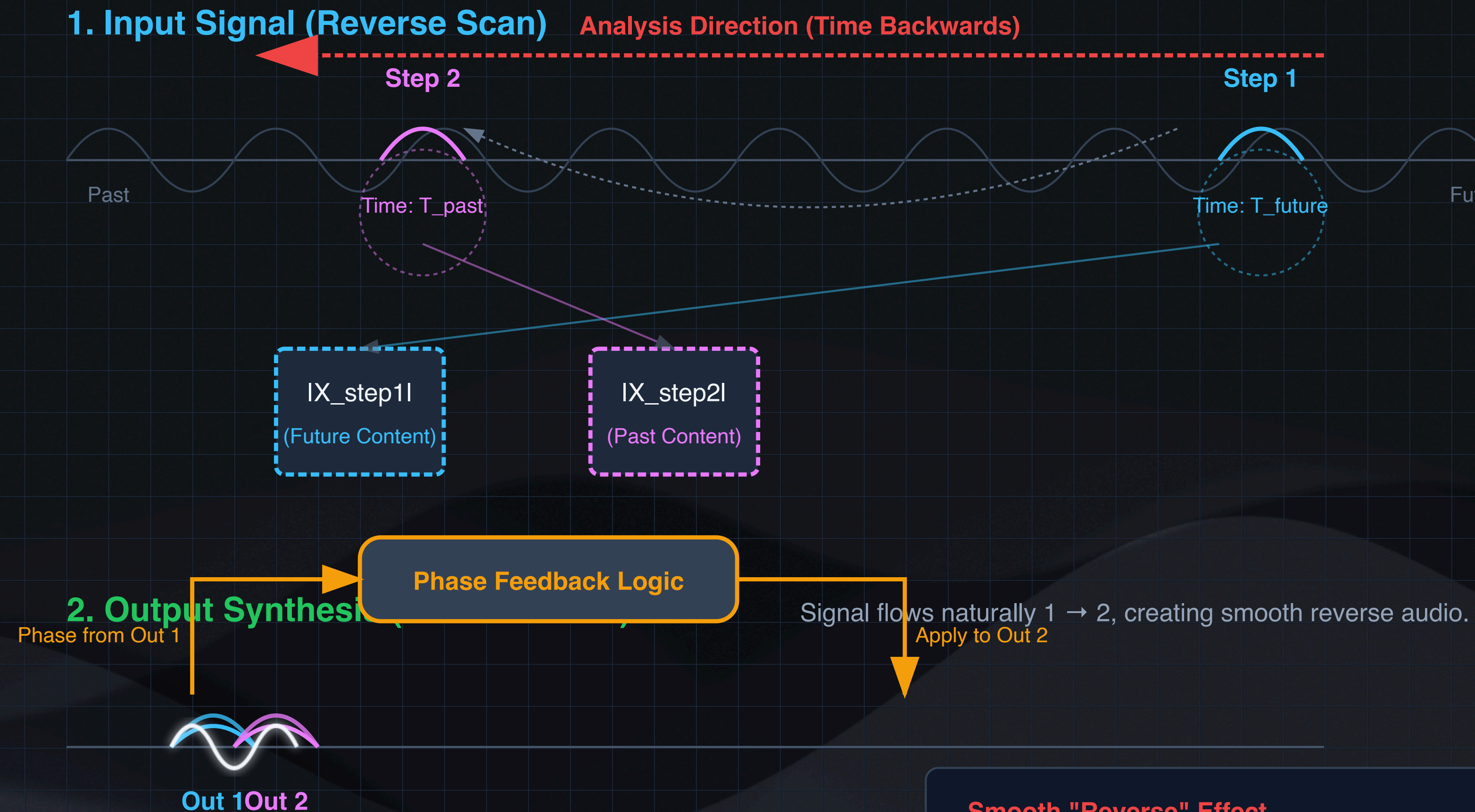
Feedback: Output Phase ensures continuity

Cross Synthesis Logic

- Multiplies spectral envelopes.
- E.g., Voice formant (A) x Synth harmonics (B)
- **Vocoder Effect with smooth phase.**

Application

Backward analysis



Smooth "Reverse" Effect

Usually, reverse playback cuts waveforms, causing "click" noises at boundaries.

Here, **Content** goes Future → Past, but **Phase** flows continuously forward.

Application

Large jump analysis

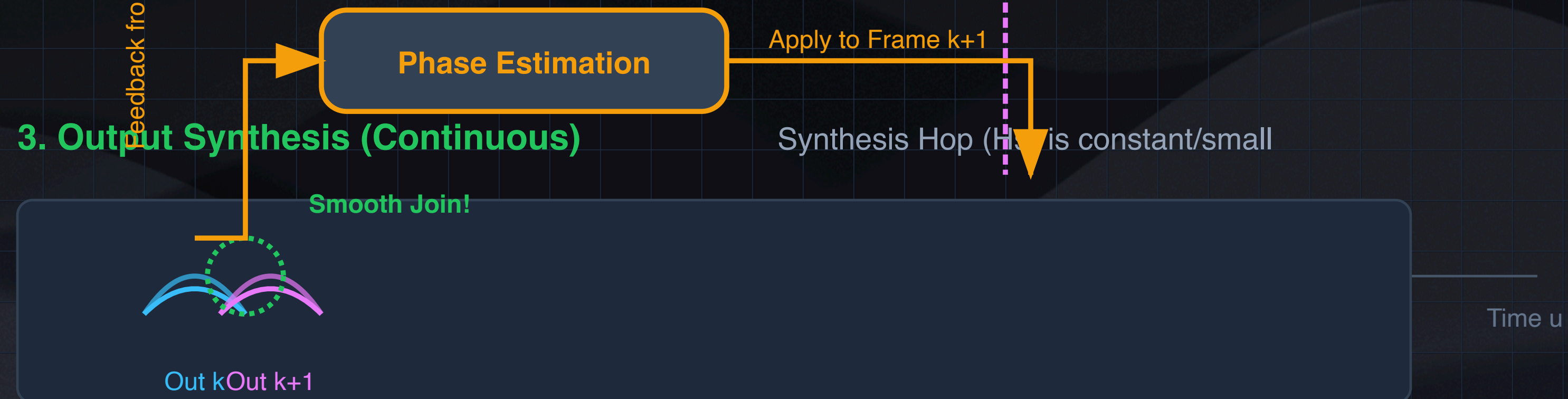
1. Input Signal (Analysis)



Key Concept

Even if input frames are far apart (e.g., Digest, Granular Synthesis), the output phase is derived from **the PREVIOUS OUTPUT frame**, ensuring perfect continuity.

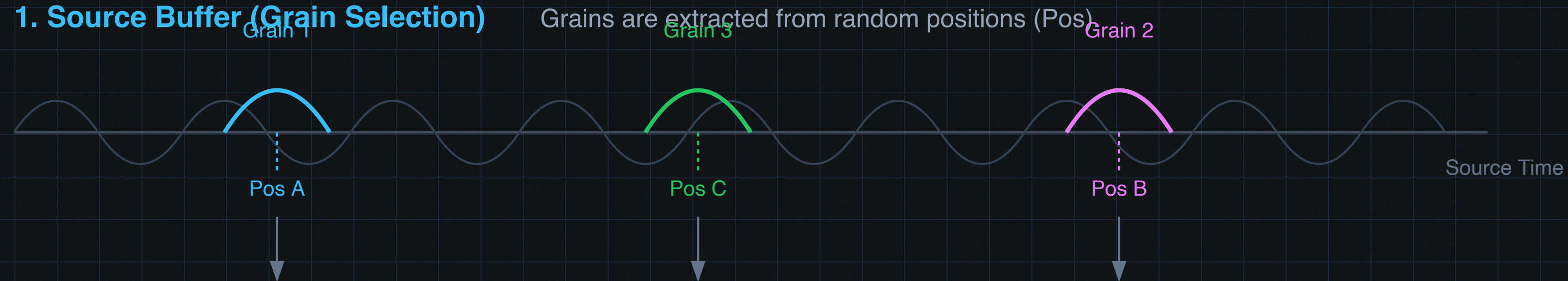
3. Output Synthesis (Continuous)



Application

Multiple frame jump analysis & synthesis (granular)

1. Source Buffer (Grain Selection)



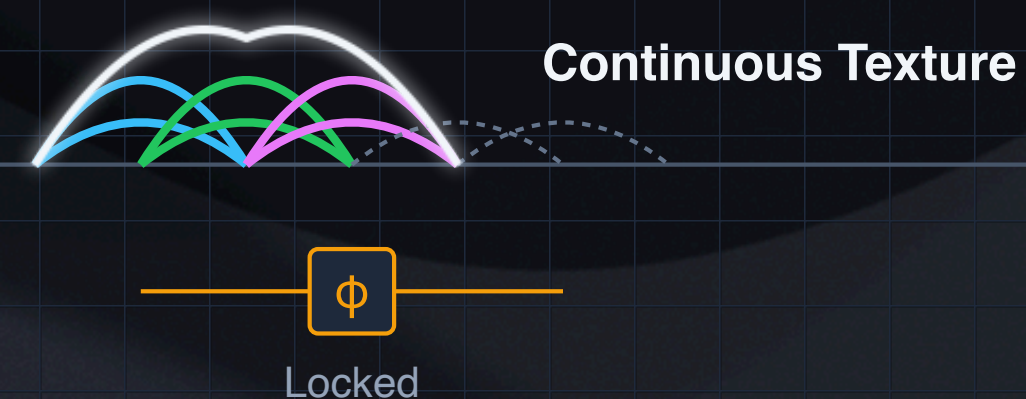
2. Phase Rectification Logic

Discard input phase, Apply coherent phase.



3. Granular Stream (Texture)

Grains overlap smoothly to form a continuous soundscape.



What is Granular Synthesis?

- Breaking sound into tiny "grains".
- Reassembling them to create new timbres or clouds.

→ Our method removes "clicks".

Future Works

- Interactive control as external object/AUv3
- Progressive phase management
 - Contrived phase rotation
 - Balancing with vertical phase coherent
- Quality improvement for tiny framewise window